

# API Testing & Quality Assurance

Buổi 8: Xây dựng chiến lược kiểm thử tự động và đo lường hiệu năng cho hệ thống

Backend Flask + MongoEngine.

---

---

# Kim tự tháp **Kiểm thử**

Từ mức Code logic (Unit) đến mức Hệ thống chịu tải (Performance).



## Unit Test

Kiểm tra các hàm độc lập. Ví dụ: Logic bấm mật khẩu, hoặc các rule Validation của MongoEngine Model.



## Integration Test

Đánh giá sự kết nối. Đảm bảo Controller giao tiếp đúng với Database (MongoDB) và trả về JSON chuẩn.



## Performance Test

Đo lường sức chịu tải (Load/Stress Testing). Tìm ra giới hạn kết nối của Flask server trước khi bị sập.



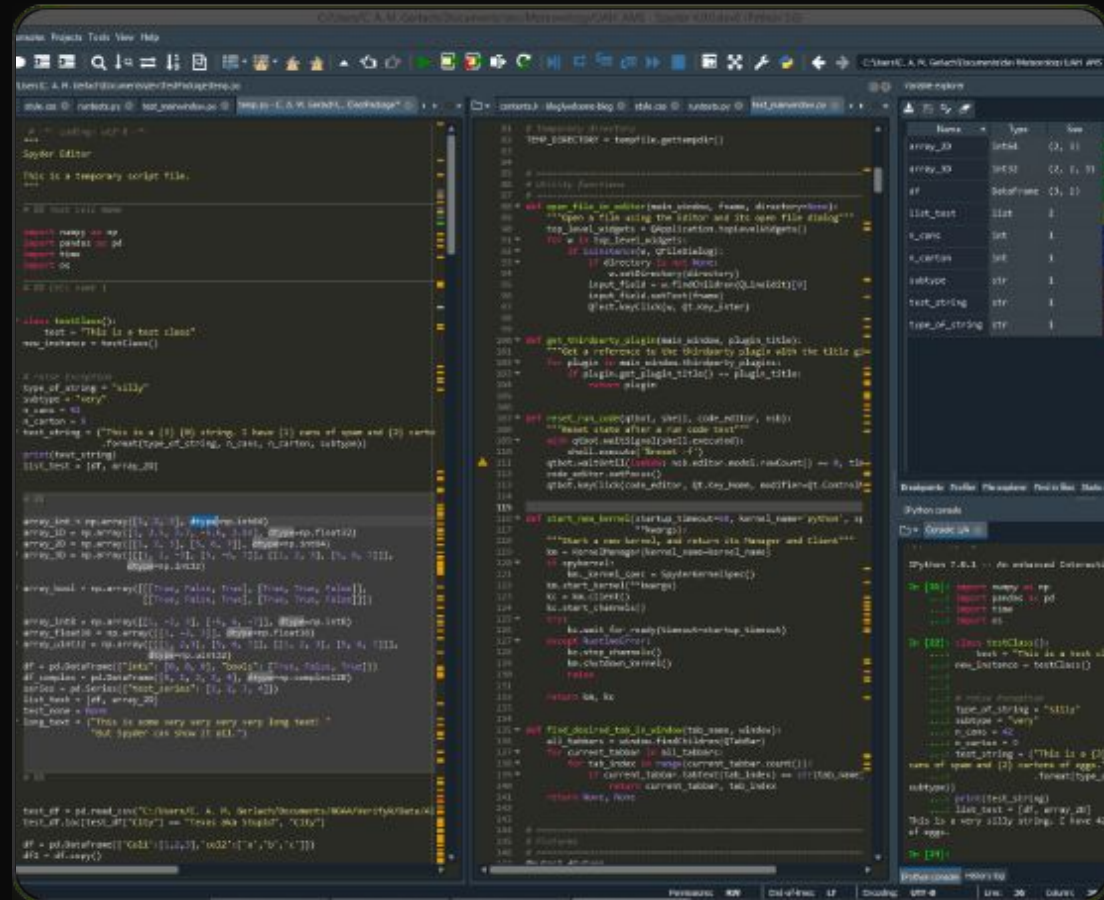
## Contract Test

Kiểm chứng API phản hồi chính xác theo bản hợp đồng OpenAPI Spec đã được thống nhất với Frontend.

# Unit Testing với Pytest

# Nền tảng của Chất lượng

- Sử dụng thư viện **pytest** kết hợp **mongomock** để giả lập Database, tránh làm bẩn dữ liệu thật.
- Tốc độ thực thi cực nhanh (vài mili-giây), giúp developer phát hiện lỗi logic ngay trong quá trình code.
- Chỉ cần viết test một lần, đảm bảo các hàm xử lý dữ liệu của MongoEngine luôn hoạt động đúng khi refactor.



---

# Tự động hóa với **Postman**

Kiểm thử End-to-End không cần giao diện người dùng.

---



## Test Scripts (Assertions)

Sử dụng JavaScript để tự động kiểm tra Status Code (200, 201) và xác thực cấu trúc dữ liệu JSON trả về.



## Environment Variables

Tự động trích xuất access\_token từ API Login và lưu vào biến môi trường để dùng cho các request sau.



## Collection Runner

Thực thi hàng loạt API theo một kịch bản định sẵn:  
Login -> Lấy sách -> Thêm sách -> Cập nhật -> Xóa.



## Newman CLI

Chạy bộ test Postman trực tiếp từ Command Line.  
Công cụ bắt buộc để tích hợp test vào luồng CI/CD

# Xây dựng Kịch bản Test

## 1. Happy Path (Kịch bản thành công)

- **POST /login:** Gửi đúng thông tin, assert status 200, lưu token.
- **POST /books:** Gửi token kèm Body hợp lệ, assert status 201 Created.
- **GET /books/{id}:** Gọi lại ID vừa tạo, assert dữ liệu trả về khớp hoàn toàn.

## 2. Negative Path (Kịch bản lỗi)

- **Gửi sai Token:** Test API xóa sách với Token giả, assert status 401 Unauthorized.
- **Thiếu trường bắt buộc:** Gửi payload tạo sách không có `title`, assert status 400 Bad Request.
- **Sai định dạng ID:** Tìm kiếm với Mongo Object ID sai, assert status 404 Not Found.

---

# Performance Testing

Đánh giá giới hạn và sức chịu tải của hệ  
thống.

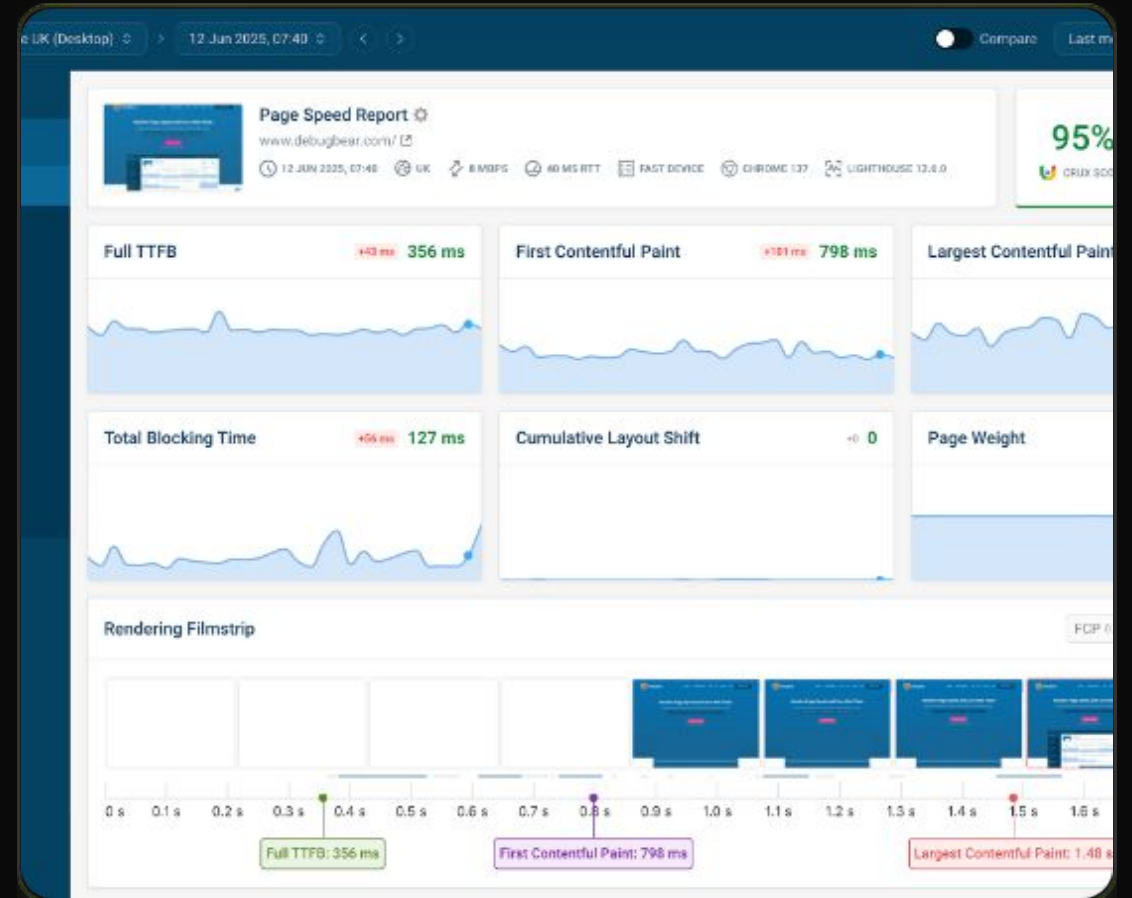
---



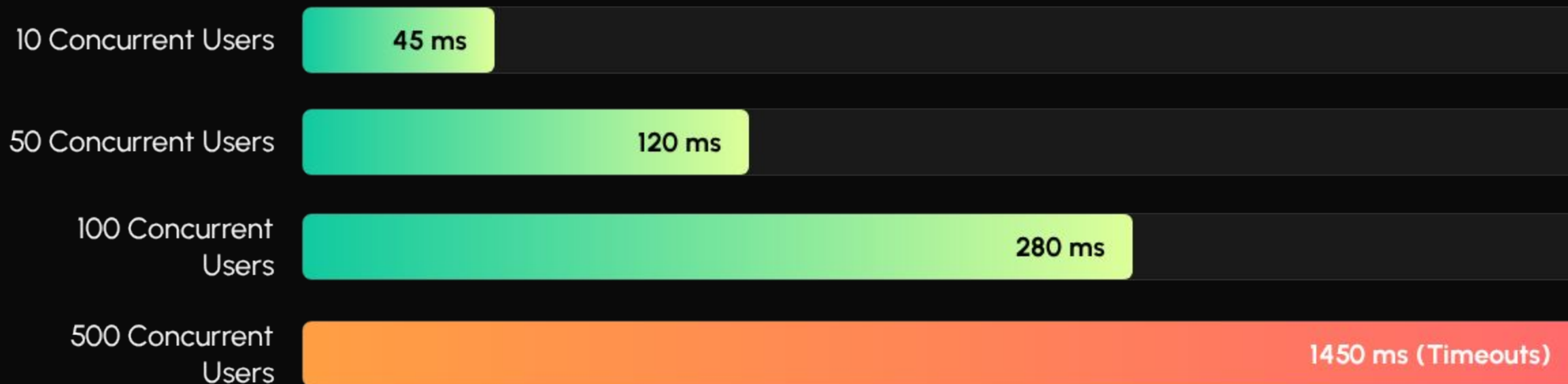
# Các chỉ số **Đo lường** cốt lõi

## Đánh giá qua Load Testing Tools (k6 / JMeter)

- **Response Time (Độ trễ):** Thời gian từ lúc Client gửi request đến lúc nhận phản hồi. Mục tiêu P95 < 200ms.
- **Error Rate (Tỷ lệ lỗi):** Phần trăm request bị thất bại (mã 5xx). Đòi hỏi < 1% dưới mức tải cao.
- **Throughput (Thông lượng):** Số lượng Requests Per Second (RPS) tối đa hệ thống xử lý được trước khi nghẽn cổ chai.



# Minh họa suy giảm Response Time



Biểu đồ Load Test giả lập trên Flask: Độ trễ tăng vọt và xuất hiện Timeout khi vượt qua ngưỡng 100 người dùng truy cập đồng thời do giới hạn xử lý đồng bộ.

# | Tích hợp QA vào vòng đời phát triển



# Q&A

Kết thúc Buổi 8: API Testing & Quality Assurance

---

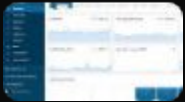
# | Image Sources



<https://user-images.githubusercontent.com/17051931/48865665-ea343480-ed95-11e8-9344-eb8df3129d12.png>

Source: [github.com](https://github.com)

---



<https://www.debugbear.com/dimg/15e4da8ef7f8dfb0c92218457780d158.png>

Source: [www.debugbear.com](https://www.debugbear.com)

---